OTIMIZAÇÃO DE MAKESPAN EM PROJETOS DE SOFTWARE: ALINHAMENTO COMPORTAMENTAL E TÉCNICO

Roberto Júnior Cassimiro Campolina
Operations Research and Complex Systems Lab (ORCS)
Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, MG, Brasil
robertojuniorme@gmail.com

André Costa Batista
Departamento de Engenharia Elétrica
Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, MG, Brasil
andre-costa@ufmg.br

RESUMO

O gerenciamento de atividades de desenvolvimento em projetos de software pode ser um desafio tático complexo, onde a alocação de tarefas a recursos humanos pode se tornar um gargalo. A alocação de tarefas em projetos de software é tradicionalmente modelada considerando apenas restrições técnicas. Este artigo propõe um modelo de Programação Quadrática Inteira Mista com Restrições (MIQCP) para o Problema de Escalonamento de Projetos de Software (SPSP) que incorpora a compatibilidade comportamental. A formulação penaliza o desalinhamento através de uma função polinomial cúbica dos perfis Solides, aliada a um bônus técnico. A resolução de 1380 instâncias no Gurobi Optimizer demonstra que o ruído cognitivo eleva o *makespan* e valida matematicamente a Lei de Brooks, fornecendo uma métrica relevante para o planejamento tático do projeto.
PALAVRAS CHAVE. Pesquisa Operacional, Programação Quadrática Inteira Mista, Engenharia de Software, Gestão Ágil, SPSP.

Tópicos: Pesquisa Operacional na Indústria, Gestão de Projetos, Otimização Combinatória.

ABSTRACT

The management of development activities in software projects can be a complex tactical challenge, where the allocation of tasks to human resources can become a bottleneck. Task allocation in software projects is traditionally modeled considering only technical constraints. This paper proposes a Mixed-Integer Quadratically Constrained Programming (MIQCP) model for the Software Project Scheduling Problem (SPSP) that incorporates behavioral compatibility. The formulation penalizes misalignment through a cubic polynomial function of Solides profiles, allied with a technical bonus. Solving 1380 instances in the Gurobi Optimizer demonstrates that cognitive noise increases the makespan and mathematically validates Brooks's Law, providing a relevant metric for the project's tactical planning.
KEYWORDS. Operations Research. Mixed-Integer Quadratic Programming. Software Engineering. Agile Management. SPSP.

Paper topics: OR in Industry. Project Management. Combinatorial Optimization.

1. Modelagem Matemática

A premissa central deste modelo é que a produtividade técnica é impactada pelo perfil do desenvolvedor. A formulação MILP proposta calcula um tempo de execução ajustado *a priori* para cada par tarefa-desenvolvedor com base em características técnicas e comportamentais combinadas.

1.1. Conjuntos, Parâmetros e Variáveis

Conjuntos e Índices

M	Conjunto de desenvolvedores, indexado por $i \in \{1, \ldots, \lvert M \rvert\}$.
N	Conjunto de tarefas no *backlog*, indexado por $j, k \in \{1, \ldots, \lvert N \rvert\}$.
S	Conjunto de *sprints* disponíveis, indexado por $s \in \{1, \ldots, \lvert S \rvert\}$.
H	Conjunto de habilidades técnicas, indexado por $h \in \{1, \ldots, \lvert H \rvert\}$.
P	Conjunto dos 4 quadrantes comportamentais, indexado por $p \in \{1, \ldots, 4\}$.
F	Conjunto de frações operacionais permitidas (em doze avos): $\{3, 4, 6, 8, 9, 12\}$.

Parâmetros

δ_s	Duração de cada *sprint* em dias corridos.
T_j	Esforço nominal (em horas) estimado para a tarefa j.
S_j	Número máximo permitido (1, 2, 3 ou 4) de desenvolvedores na tarefa j.
$\beta_j \in \{0, 1\}$	Natureza da produtividade da tarefa j, sendo 1 aditiva e 0 disjuntiva.
j_{lab}	Horas úteis que podem ser trabalhadas por dia.
Ht_{jh}	Nível técnico mínimo h exigido pela tarefa j.
Hd_{ih}	Nível técnico atual do desenvolvedor i na habilidade h.
Pt_{jp}	Fração do perfil comportamental p ideal para a tarefa j.
Pd_{ip}	Fração do perfil comportamental p do desenvolvedor i.
Cap_{is}	Capacidade disponível (horas) do desenvolvedor i na *sprint* s.
$Pred(j)$	Conjunto de tarefas que precedem estritamente a tarefa j.
E_j	Conjunto de tarefas pertencentes ao mesmo eixo temático da tarefa j.
$S_{max,j}$	Prazo limite (*deadline*) para a conclusão da tarefa j (em nível de *sprint*).
$Cerim_s$	Total de horas consumidas por ritos metodológicos na *sprint* s.
$Buffer$	Margem de segurança reservada para imprevistos (0 a 1).
Gap_{prec}	Intervalo mínimo obrigatório (horas) entre tarefas com dependência.
$Setup_{dev}$	Tempo fixo de transição operacional (horas) para o início de uma nova tarefa.

Variáveis de Decisão

$\lambda_{ijsf} \in \{0,1\}$	Variável binária de seleção que assume 1 se o dev i assumir a fração exata $f \in F$ da tarefa j na *sprint* s.
$\gamma_{ijsf} \in \mathbb{R}$	Variável contínua auxiliar de linearização exata (produto de F_{ijs}^{total} por λ_{ijsf}).
$x_{ijs} \in [0,1]$	Fração contínua de esforço, definida estritamente pela variável de seleção λ.
$z_{js} \in \{0,1\}$	1 se a tarefa j for finalizada na *sprint* s, 0 caso contrário.
$w_{ijs} \in \{0,1\}$	1 se o dev i atua na tarefa j na *sprint* s, 0 caso contrário.
$v_{ijks} \in \{0,1\}$	1 se o desenvolvedor i executa a tarefa j antes da tarefa k na *sprint* s.
$Inic_j \geq 0$	Momento exato de início da tarefa j no relógio global.
$Dur_j \geq 0$	Duração efetiva da tarefa j no calendário da sprint.
$T^{max} \geq 0$	Variável do *Makespan* total do projeto.
$u_{jsd} \in \{0,1\}$	Variável binária que assume 1 se exatamente d desenvolvedores ($d \in \{1,\ldots,S_j\}$) forem alocados à tarefa j na *sprint* s.
$y_{ijsd} \in \{0,1\}$	Variável binária de apoio para linearização, representando o produto lógico entre w_{ijs} e u_{jsd}.

1.2. Parâmetros Adicionais e Variáveis Calculadas (Pré-Processamento)

A verificação de competência técnica passa a ser tratada como um filtro booleano calculado antes da resolução, simplificando o processamento do otimizador:

Matriz de Suficiência Técnica (HdT_{ijh})
Um parâmetro binário que indica se o desenvolvedor i possui o domínio exigido pela tarefa j na habilidade h. Ele assume valor 1 se $Hd_{ih} \geq Ht_{jh}$, e 0 caso contrário.

$$HdT_{ijh} = \begin{cases} 1 & \text{se } Hd_{ih} \geq Ht_{jh} \\ 0 & \text{caso contrário} \end{cases} \quad \forall i \in M, \forall j \in N, \forall h \in H \tag{1}$$

Cálculo da Matriz de Tempo

O ajuste numérico do tempo de execução transita por duas fases. A primeira fase consiste no pré-processamento estático das variáveis individuais, estruturada em três etapas que quantificam o impacto das variáveis humanas na produtividade base:

Etapa 1: Excedente Técnico (ET_{ij})
Avalia a qualificação técnica excedente do recurso atribuído. Se o desenvolvedor superar o requisito mínimo estipulado, a execução da tarefa é acelerada. Contudo, em consonância com o princípio da Complexidade Essencial de Brooks [1987] e a saturação da Capacidade do Programador (PCAP) do modelo COCOMO II [Boehm et al., 2000], o ganho de produtividade obedece à Lei dos Rendimentos Decrescentes. Um desenvolvedor altamente sênior executando uma tarefa júnior não reduzirá o tempo a zero, pois a formulação lógica da atividade possui um limite basal de esforço.

Para modelar essa saturação de forma estritamente convexa, a diferença absoluta entre a habilidade do desenvolvedor (Hd) e a exigida pela tarefa (Ht) é elevada ao quadrado. O termo $|H| \cdot 25$ é introduzido no denominador como fator de normalização, pressupondo uma escala de proficiência baseada em 5 níveis (onde o desvio máximo ao quadrado resulta em 25). Isso garante que o multiplicador represente a proficiência média ponderada e convirja para o intervalo contínuo $[0,1]$. Finalmente, utiliza-se um coeficiente de amortização de 0.5 para garantir que o bônus técnico atue como um redutor máximo de 50% do tempo nominal, respeitando os limites biológicos de velocidade de digitação e raciocínio:

$$ET_{ij} = 0.5 \cdot \left[\frac{1}{|H| \cdot 25} \sum_{h \in H} (\max(0, Hd_{ih} - Ht_{jh}))^2 \right] \quad \forall i \in M, \forall j \in N \tag{2}$$

$$F_{tech,ij} = 1 - ET_{ij} \quad \forall i \in M, \forall j \in N \tag{3}$$

Etapa 2: Erro Comportamental (EC_{ij})
Para quantificar o desgaste cognitivo provocado pelo desalinhamento psicológico, o modelo fundamenta-se na Teoria da Carga Cognitiva [Sweller, 1988] e no conceito de *Person-Job Fit* aplicado à engenharia de software [Acuña et al., 2006; Gorla e Lam, 2004]. Calcula-se inicialmente a diferença entre os quadrantes do perfil comportamental requerido pela tarefa e o perfil real do desenvolvedor, denotada por ψ_{ij}. Esta métrica é obtida pela Norma L_1 Normalizada (Distância de Manhattan), onde a divisão por 2 atua como fator de normalização: como os perfis são representados por distribuições proporcionais (soma vetorial unitária), a maior diferença absoluta possível resulta em 2.0. Esta operação garante que ψ_{ij} convirja estritamente para o intervalo $[0, 1]$.

Diferente da competência técnica, a literatura de fatores humanos indica que o atrito cognitivo não possui comportamento linear. Pequenas discrepâncias de perfil tendem a ser mitigadas por mecanismos de adaptação social, enquanto desalinhamentos severos provocam uma "Escalada de Conflito" que compromete o estado de *Flow*. Para modelar essa sensibilidade, aplica-se uma função polinomial de ordem n com um multiplicador de ganho k. Estes parâmetros foram definidos empiricamente em fase preliminar como $n = 3$ e $k = 5.0$, garantindo que a penalidade seja desprezível em desvios baixos e crítica em oposições de perfil:

$$\psi_{ij} = \frac{1}{2} \sum_{p \in P} |Pd_{ip} - Pt_{jp}| \quad \forall i \in M, \forall j \in N \tag{4}$$

$$EC_{ij} = 5.0 \cdot (\psi_{ij})^3 \quad \forall i \in M, \forall j \in N \tag{5}$$

$$F_{beh,ij} = 1 + EC_{ij} \quad \forall i \in M, \forall j \in N \tag{6}$$

Etapa 3: Tempo Base Ajustado (T_{ij})
Diferente dos fatores dinâmicos que exigem agregação aditiva para evitar a explosão combinatória do otimizador, o Erro Comportamental (EC_{ij}) e o Erro Técnico (ET_{ij}) são processados estaticamente *a priori*. Portanto, em alinhamento com a arquitetura de *Effort Multipliers* do COCOMO II, eles incidem sobre o tempo base de forma multiplicativa, capturando o efeito composto e proporcional entre proficiência e alinhamento psicológico.

$$T_{ij} = T_j \cdot F_{beh,ij} \cdot F_{tech,ij} \quad \forall i \in M, \forall j \in N \tag{7}$$

1.3. Fatores Dinâmicos de Otimização Ágil

A segunda fase ocorre de forma endógena durante a resolução do modelo MILP. O tempo estático T_{ij} sofre interferências diretas das escolhas de alocação do *solver*, ativando os multiplicadores de fluxo de trabalho:

1. Limite de Weinberg e Troca de Contexto (F_{ctx}):
A literatura de engenharia de software [Weinberg, 1992] estabelece uma perda de aproximadamente 20% da capacidade produtiva para cada tarefa adicional assumida simultaneamente. Para garantir a viabilidade, o modelo impõe um limite estrito de 5 tarefas simultâneas:

$$\sum_{j \in N} w_{ijs} \leq 5 \quad \forall i \in M, \forall s \in S \tag{8}$$

O multiplicador de tempo resultante desta degradação aplica um acréscimo linear de 20% para cada tarefa extra, utilizando a função de máximo para garantir que a penalidade só seja ativada em cenários de multitarefa:

$$F_{ctx,is} = 0.20 \cdot \max\left(0, \sum_{j \in N} w_{ijs} - 1\right) \quad \forall i \in M, \forall s \in S \tag{9}$$

2. Lei de Brooks e Overhead de Comunicação ($F_{com,js}$):
A alocação de múltiplos desenvolvedores em uma mesma atividade gera atritos de coordenação. Segundo a Lei de Brooks [Brooks, 1975], o esforço de comunicação cresce de forma combinatória em função do número de conexões interpessoais necessárias. O total de canais de comunicação para uma equipe de tamanho d é dado por $\frac{d(d-1)}{2}$. Para calibrar a penalidade por canal (ω), o modelo adota $\omega = 0.05$ (5%), valor empiricamente fundamentado pelos estudos clássicos de estimativa de custo de Boehm [1981]. Utilizando a variável binária de discretização de equipe (u_{jsd}), que será detalhada na sessão 1.3.5 (Linearização do Match Comportamental da Equipe), o multiplicador de *overhead* é formulado preservando a linearidade do otimizador:

$$F_{com,js} = 0.05 \cdot \sum_{d=1}^{S_j} \left(\frac{d(d-1)}{2}\right) \cdot u_{jsd} \quad \forall j \in N, \forall s \in S \tag{10}$$

3. Curva de Aprendizado e Retenção de Contexto ($F_{apr,ijs}$):
A eficiência produtiva é modelada como uma variável dinâmica sensível ao histórico técnico (*Language and Tool Experience* - LTEX). Para este modelo, os parâmetros foram definidos como $\alpha_{eixo} = 0,05$ e $\lambda = 0,8$ para o domínio temático, e $\alpha_{dep} = 0,2$ e $\lambda_{dep} = 0,6$ para a precedência tecnológica. A fundamentação empírica para a calibração destes coeficientes, baseada na convergência para faixas de interesse biológico e operacional, será detalhada na seção de resultados (Seção 4).

A formulação lineariza os ganhos através de dois vetores distintos: o primeiro trata da consolidação estrutural em um domínio de negócio (α_{eixo}), utilizando o fenômeno da Curva do Esquecimento [Ebbinghaus, 1885] com uma taxa de retenção λ sobre o hiato de *sprints* ($\Delta s = s - p$):

$$B_{ijs}^{eixo} = 0,05 \sum_{p=1}^{s-1} \sum_{k \in E_j} w_{ikp} \cdot 0,8^{(s-p)} \quad \forall i \in M, \forall j \in N, \forall s \in S \tag{11}$$

O segundo vetor trata da memória de trabalho recente (*Working Memory*). O contexto arquitetural é mais forte nas adjacências diretas da cadeia de dependência. Seja $Pred^*(j)$ o fecho transitivo de predecessoras da tarefa j, e $dist(k, j)$ a distância no grafo entre

a predecessora k e a tarefa j. O bônus acumula-se através de uma progressão geométrica com fator de decaimento de profundidade $\lambda_{dep} = 0,6$:

$$B_{ij}^{dep} = 0,2 \sum_{k \in Pred^*(j)} \left(0,6^{dist(k,j)-1}\right) \sum_{p \in S} w_{ikp} \quad \forall i \in M, \forall j \in N \tag{12}$$

O fator dinâmico final de aprendizado é dado pela soma das componentes:

$$F_{apr,ijs} = B_{ijs}^{eixo} + B_{ij}^{dep} \quad \forall i \in M, \forall j \in N, \forall s \in S \tag{13}$$

4. Teoria da Carga Cognitiva e Fadiga ($F_{\Phi,is}$):
A exaustão em projetos de software transcende o esforço físico cronometrado. Segundo a Teoria da Carga Cognitiva de Sweller [1988], a execução de uma tarefa impõe uma carga intrínseca, que é severamente agravada por cargas extrínsecas (como o atrito psicológico de atuar em atividades desalinhadas ao perfil).

Para operacionalizar matematicamente este fenômeno mantendo a linearidade do otimizador, a formulação proposta calcula o Índice de Carga Cognitiva ($Cg_{\%,is}$), variando de 0% a 100%. Este índice divide o esforço agregado do desenvolvedor pela Carga Máxima Teórica do modelo ($Cg_{max} = 30$, derivado do limite de 5 tarefas simultâneas de Weinberg sob a penalidade comportamental máxima de 6.0):

$$Cg_{\%,is} = \frac{\sum_{j \in N} w_{ijs} \cdot F_{beh,ij}}{30} \quad \forall i \in M, \forall s \in S \tag{14}$$

Em alinhamento com o princípio de Ritmo Sustentável (*Sustainable Pace*) de Beck [1999], o desempenho humano mantém-se estável apenas até um Limiar de Fadiga tolerável (L_{fat}). Caso o índice cognitivo ultrapasse este limiar de ruptura, o modelo ativa um multiplicador dinâmico de degradação *piecewise linear* (linear por partes), mediado por uma taxa de exaustão (ϕ). Para a presente formulação, os parâmetros foram definidos como $L_{fat} = 0,10$ e $\phi = 9,0$. A justificativa e demonstração empírica para a calibração destes valores será detalhada na seção de resultados (Seção 4). A equação final aplicada é dada por:

$$F_{\Phi,is} = 9,0 \cdot \max(0, Cg_{\%,is} - 0,10) \quad \forall i \in M, \forall s \in S \tag{15}$$

5. Linearização do Match Comportamental da Equipe

A formulação original do *TeamMatch* exigia a divisão da soma do Erro Comportamental (EC) pelo total de desenvolvedores alocados ($\sum w_{ijs}$), caracterizando uma não-linearidade. Como o tamanho da equipe assume valores estritamente inteiros $d \in \{1, \ldots, S_j\}$, introduzimos a técnica de discretização do denominador.

5.1. Novas Variáveis Auxiliares:

$u_{jsd} \in \{0,1\}$	1 se exatamente d desenvolvedores estão alocados na tarefa j na *sprint* s, 0 caso contrário.
$y_{ijsd} \in \{0,1\}$	Variável de linearização que representa o produto lógico entre w_{ijs} e u_{jsd}.

5.2. Contagem Discreta da Equipe e Vínculo de Esforço:
Garante que, se a tarefa for executada na *sprint* ($z_{js} = 1$), exatamente uma variável de tamanho de equipe (u_{jsd}) será ativada. Em seguida, vincula-se essa escolha à soma de

desenvolvedores alocados. Por fim, para evitar a fragmentação ilusória, obriga-se que a participação na equipe (w_{ijs}) seja estritamente condicionada à adoção de uma fração real de esforço (λ_{ijsf}):

$$\sum_{d=1}^{S_j} u_{jsd} = z_{js} \quad \forall j \in N, \forall s \in S \tag{16}$$

$$\sum_{i \in M} w_{ijs} = \sum_{d=1}^{S_j} (d \cdot u_{jsd}) \quad \forall j \in N, \forall s \in S \tag{17}$$

$$w_{ijs} = \sum_{f \in F} \lambda_{ijsf} \quad \forall i \in M, \forall j \in N, \forall s \in S \tag{18}$$

5.3. Linearização do Produto (*Glover's Linearization*):
Como precisamos multiplicar EC_{ij} pelo fato de o desenvolvedor i estar na equipe ($w_{ijs} = 1$) E o tamanho da equipe ser d ($u_{jsd} = 1$), criamos a variável $y_{ijsd} = w_{ijs} \cdot u_{jsd}$. As equações abaixo aplicam a técnica clássica de linearização para o produto de variáveis binárias [Glover, 1975; Watters, 1967], garantindo que y só assuma valor 1 se ambas as variáveis originais forem 1:

$$y_{ijsd} \leq w_{ijs} \quad \forall i \in M, \forall j \in N, \forall s \in S, \forall d \in \{1, \ldots, S_j\} \tag{19}$$

$$y_{ijsd} \leq u_{jsd} \quad \forall i \in M, \forall j \in N, \forall s \in S, \forall d \in \{1, \ldots, S_j\} \tag{20}$$

$$y_{ijsd} \geq w_{ijs} + u_{jsd} - 1 \quad \forall i \in M, \forall j \in N, \forall s \in S, \forall d \in \{1, \ldots, S_j\} \tag{21}$$

5.4. Sinergia Interpessoal e Balanceamento da Equipe ($F_{match,js}$):
A literatura de engenharia de software [Gorla e Lam, 2004; Acuña et al., 2006] amparada na Teoria de Papéis de Equipe [Belbin, 1993], demonstra empíricamente que equipes de desenvolvimento com perfis psicológicos heterogêneos superam equipes homogêneas, pois a diversidade cognitiva mitiga "pontos cegos" e conflitos de dominância.

Para avaliar a compatibilidade interpessoal baseada nos perfis dos desenvolvedores (Pd_{ip}), o modelo calcula o Perfil Centroide da Equipe (Peq_{jsp}). Uma equipe perfeitamente complementar possui uma distribuição equitativa de 25% (0.25) em cada um dos quatro quadrantes do perfil Solides. A variável auxiliar contínua $Dev_{jsp} \geq 0$ mede o desvio absoluto da equipe em relação a esse equilíbrio ótimo.

Para evitar falsas penalidades em alocações individuais, o cálculo só é ativado para equipes com múltiplos integrantes ($d \geq 2$), utilizando a variável linearizada y_{ijsd} pré-existente:

$$Peq_{jsp} = \sum_{i \in M} \sum_{d=2}^{S_j} \left(\frac{Pd_{ip}}{d} \right) \cdot y_{ijsd} \quad \forall j \in N, \forall s \in S, \forall p \in P \tag{22}$$

A linearização do módulo do desvio (distância absoluta para o alvo de 0.25) é obtida pelo par de inequações, onde a constante ideal é condicionada à existência da equipe para evitar penalidades em tarefas não executadas:

$$Dev_{jsp} \geq Peq_{jsp} - 0.25 \cdot \sum_{d=2}^{S_j} u_{jsd} \quad \forall j \in N, \forall s \in S, \forall p \in P \tag{23}$$

$$Dev_{jsp} \geq 0.25 \cdot \sum_{d=2}^{S_j} u_{jsd} - Peq_{jsp} \quad \forall j \in N, \forall s \in S, \forall p \in P \tag{24}$$

A adoção do desvio absoluto (Norma L_1) em detrimento da variância estatística (L_2) é uma escolha de *design* fundamental para reduzir a dimensionalidade não-linear do modelo, limitando a formulação quadrática (MIQCP) estritamente à etapa final de cálculo de esforço.

O fator de sinergia final penaliza o *makespan* proporcionalmente ao desbalanceamento interpessoal. O somatório dos desvios atinge seu valor máximo de 1.5 em equipes completamente homogêneas. Utilizando o termo divisor 1.5 para normalizar a métrica para o intervalo contínuo $[0, 1]$, define-se um peso de penalidade máxima θ. O parâmetro θ representa o limite superior de degradação temporal causado por atritos sociológicos extremos. Baseado nas observações sobre dinâmica de equipes disfuncionais de DeMarco e Lister [DeMarco e Lister, 1987], define-se $\theta = 0.50$ (indicando até 50% de perda em equipes de incompatibilidade máxima).

$$F_{match,js} = 0.5 \cdot \left(\frac{\sum_{p \in P} Dev_{jsp}}{1.5} \right) \quad \forall j \in N, \forall s \in S \tag{25}$$

6. Tempo Dinâmico Final de Execução (T_{ijs}^{din}):

Para maior clareza matemática, o cálculo do tempo dinâmico de execução é desmembrado em duas etapas. A primeira etapa consolida o tempo auxiliar de esforço (T_{ijs}^{aux}), partindo da parcela base de trabalho ($T_{ij} \cdot x_{ijs}$). Para garantir que a formulação permaneça estritamente linear (MILP) e evitar a explosão combinatória, o cálculo do tempo dinâmico utiliza a técnica de linearização *Big-M* exata para produtos de variáveis contínuas e binárias [Glover, 1975]. Como a alocação de esforço foi discretizada através da variável binária de seleção (λ_{ijsf}), a multiplicação não-linear ocorre exclusivamente entre uma variável contínua (o fator total de oscilação) e uma binária.

Primeiramente, consolida-se a carga basal do tempo nominal (representada pela unidade constante) com os fatores dinâmicos aditivos na variável contínua F_{ijs}^{total}:

$$F_{ijs}^{total} = 1 + F_{ctx,is} + F_{com,js} + F_{\Phi,is} + F_{match,js} - F_{apr,ijs} \quad \forall i \in M, \forall j \in N, \forall s \in S \tag{26}$$

Em seguida, introduz-se a variável contínua γ_{ijsf}, que representa o produto exato de $F_{ijs}^{total} \cdot \lambda_{ijsf}$. A linearização é garantida por quatro restrições *Big-M*. Define-se o limite inferior $M^{min} = -1000$ e superior $M^{max} = 1000$. A arquitetura atua como um desvio condicional (*if/else*): se a fração não for selecionada ($\lambda_{ijsf} = 0$), as restrições 31 e 32 forçam $\gamma_{ijsf} = 0$. Caso a fração seja selecionada ($\lambda_{ijsf} = 1$), as restrições 33 e 34 forçam γ_{ijsf} a assumir exatamente o valor do fator total (F_{ijs}^{total}):

$$\gamma_{ijsf} \leq M^{max} \cdot \lambda_{ijsf} \quad \forall i \in M, \forall j \in N, \forall s \in S, \forall f \in F \tag{27}$$

$$\gamma_{ijsf} \geq M^{min} \cdot \lambda_{ijsf} \quad \forall i \in M, \forall j \in N, \forall s \in S, \forall f \in F \tag{28}$$

$$\gamma_{ijsf} \leq F_{ijs}^{total} - M^{min} \cdot (1 - \lambda_{ijsf}) \quad \forall i \in M, \forall j \in N, \forall s \in S, \forall f \in F \tag{29}$$

$$\gamma_{ijsf} \geq F_{ijs}^{total} - M^{max} \cdot (1 - \lambda_{ijsf}) \quad \forall i \in M, \forall j \in N, \forall s \in S, \forall f \in F \tag{30}$$

Finalmente, o tempo auxiliar de esforço (T_{ijs}^{aux}) é calculado de forma puramente linear, multiplicando-se o esforço basal pelo somatório isolado da variável linearizada:

$$T_{ijs}^{aux} = T_{ij} \cdot \left[\frac{1}{12} \sum_{f \in F} (f \cdot \gamma_{ijsf}) \right] \quad \forall i \in M, \forall j \in N, \forall s \in S \tag{31}$$

Onde os fatores dinâmicos atuam como osciladores proporcionais ao tempo nominal. Na segunda etapa, o tempo dinâmico final (T_{ijs}^{din}) é obtido pela soma do esforço auxiliar com o custo fixo de transição (*setup*):

Na segunda etapa, o tempo dinâmico final (T_{ijs}^{din}) é obtido pela soma do esforço auxiliar com o custo fixo de transição (*setup*), que é cobrado integralmente sempre que ocorre a alocação da tarefa ao desenvolvedor:

$$T_{ijs}^{din} = T_{ijs}^{aux} + w_{ijs} \cdot Setup_{dev} \quad \forall i \in M, \forall j \in N, \forall s \in S \tag{32}$$

A literatura empírica de engenharia de software, notadamente McConnell [1996] e DeMarco e Lister [DeMarco e Lister, 1987], evidencia a existência de um período inicial de adaptação cognitiva (*ramp-up intra-tarefa*) antes que a produtividade atinja seu platô ótimo. Para incorporar este fenômeno empírico sem incorrer na explosão combinatória dimensional provocada por variáveis discretas diárias no otimizador MILP, o modelo abstrai a curva de aceleração diária através do parâmetro isolado $Setup_{dev}$. Este custo fixo atua como a penalidade cognitiva de imersão inicial, garantindo a fidelidade biológica do escalonamento mantendo a resolução estritamente linear.

Duração Real no Calendário (Paralelismo):
Como o modelo permite que múltiplos desenvolvedores atuem simultaneamente em uma mesma tarefa (paralelismo operacional), o tempo consumido no calendário da *sprint* não é a soma dos esforços da equipe, mas sim o esforço do integrante que finalizar sua parte por último. Para capturar este gargalo temporal, introduz-se a variável de duração contínua (Dur_j), regida pela inequação limitante que força a duração da tarefa a ser, no mínimo, igual ao maior esforço individual gasto nela:

$$Dur_j \geq T_{ijs}^{din} \quad \forall i \in M, \forall j \in N, \forall s \in S \tag{33}$$

1.4. Função Objetivo e Restrições

O modelo minimiza o cronograma final de entrega do projeto, representado pelo *makespan*:

$$\min T^{max} \tag{34}$$

A alocação das tarefas está sujeita às restrições operacionais e lógicas descritas a seguir:

1. Conclusão Integral do Backlog:
Garante que cada tarefa j atinja 100% de execução através da soma das frações de esforço alocadas na equipe:

$$\sum_{s \in S} \sum_{i \in M} \sum_{f \in F} (f \cdot \lambda_{ijsf}) = 12 \quad \forall j \in N \tag{35}$$

2. Atomicidade da Sprint (Fim do Carry-over):

Impede que uma tarefa seja fragmentada entre ciclos. O esforço total da tarefa j deve ser integralmente alocado a uma única *sprint* s, respeitando os limites temporais da janela (δ_s):

$$z_{js} = \sum_{i \in M} x_{ijs} \quad \forall j \in N, \forall s \in S \tag{36}$$

$$\sum_{s \in S} z_{js} = 1 \quad \forall j \in N \tag{37}$$

$$Inic_j \geq \sum_{s \in S} ((s-1) \cdot \delta_s) \cdot z_{js} \quad \forall j \in N \tag{38}$$

$$Inic_j + Dur_j \leq \sum_{s \in S} (s \cdot \delta_s \cdot 8) \cdot z_{js} \quad \forall j \in N \tag{39}$$

3. Respeito aos Prazos Limites (Deadlines):
Obriga a conclusão da tarefa em uma *sprint* igual ou anterior à data limite:

$$\sum_{s \in S} s \cdot z_{js} \leq S_{max,j} \quad \forall j \in N \tag{40}$$

4. Suficiência Técnica da Equipe:
Garante que pelo menos um dos desenvolvedores alocados (w_{ijs}) possua o nível de habilidade mínimo exigido pela tarefa:

$$\sum_{i \in M} \sum_{s \in S} HdT_{ijh} \cdot w_{ijs} \geq 1 \quad \forall j \in N, \forall h \in H \text{ onde } Ht_{jh} > 0 \tag{41}$$

5 e 6. Limites de Equipe e Discretização do Esforço:
Para conferir maior realismo à gestão do projeto e evitar a fragmentação microscópica do esforço, o modelo restringe a alocação (x_{ijs}) a um conjunto estrito de frações operacionais típicas. Utilizando o Mínimo Múltiplo Comum para evitar dízimas periódicas, a tarefa é dividida em 12 fragmentos indivisíveis. Define-se o conjunto de opções permitidas $F = \{3, 4, 6, 8, 9, 12\}$ (que representam 25%, 33%, 50%, 66%, 75% e 100% do esforço). A variável binária de seleção (λ_{ijsf}) garante que o otimizador não assuma valores intermediários, enquanto o limite simultâneo da equipe (S_j) é cravado em no máximo 4 integrantes:

$$\sum_{i \in M} w_{ijs} \leq 4 \quad \forall j \in N, \forall s \in S \tag{42}$$

$$w_{ijs} = \sum_{f \in F} \lambda_{ijsf} \quad \forall i \in M, \forall j \in N, \forall s \in S \tag{43}$$

$$x_{ijs} = \frac{1}{12} \sum_{f \in F} (f \cdot \lambda_{ijsf}) \quad \forall i \in M, \forall j \in N, \forall s \in S \tag{44}$$

7 e 8. Dependência Cronológica e Validação (Code Review):
A tarefa sucessora j só inicia após a conclusão integral da predecessora k somada ao tempo de validação e testes:

$$Inic_j \geq Inic_k + Dur_k + Gap_{prec} \quad \forall j \in N, \forall k \in Pred(j) \tag{45}$$

9. Sequenciamento de Desenvolvedor (Evitar Sobreposição):
Garante que um desenvolvedor não execute tarefas simultaneamente na mesma *sprint*. As equações (21) e (22) forçam a escolha de uma ordem cronológica, enquanto as equações (23) e (24) atuam como travas de segurança, garantindo que o sequenciamento só ocorra se ambas as tarefas forem efetivamente atribuídas:

$$v_{ijks} + v_{ikjs} \geq w_{ijs} + w_{iks} - 1 \quad \forall i \in M, \forall s \in S, \forall j, k \in N \ (j \neq k) \tag{46}$$

$$Inic_k \geq Inic_j + T^{din}_{ijs} - \mathcal{M} \cdot (1 - v_{ijks}) \quad \forall i \in M, \forall s \in S, \forall j, k \in N \ (j \neq k) \tag{47}$$

$$v_{ijks} + v_{ikjs} \leq w_{ijs} \quad \forall i \in M, \forall s \in S, \forall j, k \in N \ (j \neq k) \tag{48}$$

$$v_{ijks} + v_{ikjs} \leq w_{iks} \quad \forall i \in M, \forall s \in S, \forall j, k \in N \ (j \neq k) \tag{49}$$

10. Limite da Carga Horária (Capacidade Útil):
O esforço dinâmico somado de todas as tarefas não pode exceder o limite produtivo real do desenvolvedor. A Capacidade Útil ($CapUtil_{is}$) é pré-calculada subtraindo-se as horas consumidas por ritos metodológicos ($Cerim_s$) da capacidade nominal disponível (Cap_{is}), aplicando-se em seguida a margem de segurança contra imprevistos ($Buffer$):

$$CapUtil_{is} = (Cap_{is} - Cerim_s) \cdot (1 - Buffer) \quad \forall i \in M, \forall s \in S \tag{50}$$

$$\sum_{j \in N} T^{din}_{ijs} \leq CapUtil_{is} \quad \forall i \in M, \forall s \in S \tag{51}$$

Fechamento do Projeto:
O *Makespan* é delimitado pelo fim (Início + Duração) da última tarefa do backlog:

$$T^{max} \geq Inic_j + Dur_j \quad \forall j \in N \tag{52}$$

Referências

Acuña, S. T., Gómez, M., e Juristo, N. (2006). How do personality, team processes, and task characteristics relate to job satisfaction and software quality? *Information and Software Technology*, 48(8):622–636.

Beck, K. (1999). *Extreme Programming Explained: Embrace Change*. Addison-Wesley Professional.

Belbin, R. M. (1993). *Team Roles at Work*. Butterworth-Heinemann.

Boehm, B. W. (1981). *Software Engineering Economics*. Prentice-Hall, Englewood Cliffs, NJ.

Boehm, B. W., Abts, C., Brown, A. W., Chulani, S., Clark, B. K., Horowitz, E., Madachy, R., Reifer, D. J., e Steece, B. (2000). *Software Cost Estimation with COCOMO II*. Prentice Hall.

Brooks, F. P. (1975). *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley Professional.

Brooks, F. P. (1987). No silver bullet: Essence and accidents of software engineering. *IEEE Computer*, 20(4):10–19.

DeMarco, T. e Lister, T. (1987). *Peopleware: Productive Projects and Teams*. Dorset House Publishing Co.

Ebbinghaus, H. (1885). *Memory: A Contribution to Experimental Psychology*. Teachers College, Columbia University. Tradução para o inglês em 1913 por H. A. Ruger e C. E. Bussenius.

Glover, F. (1975). Improved linear integer programming formulations of nonlinear integer problems. *Management Science*, 22(4):455–460.

Gorla, N. e Lam, Y. W. (2004). Who should work with whom? building effective software teams. *Communications of the ACM*, 47(6):79–82.

McConnell, S. (1996). *Rapid Development: Taming Wild Software Schedules*. Microsoft Press.

Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285.

Watters, L. J. (1967). Reduction of integer polynomial programming problems to zero-one linear programming problems. *Operations Research*, 15(6):1171–1174.

Weinberg, G. M. (1992). *Quality Software Management: Systems Thinking*, volume 1. Dorset House Publishing, New York, NY.